

AgKalkan: C++ and Qt-Based Object-Oriented Network Reverse Engineering and Behavioral Threat Detection System

Elnur Azizov

Computer Engineering

Kastamonu University, Kastamonu / Türkiye

azizovelnur2@gmail.com

ORCID: 0009-0005-1087-3838

Abstract— When taking cybersecurity courses, the first tools we encounter are usually ready-made network monitoring software. As students, we look at the colorful lines provided by these tools, but we are mostly unable to see how those hexadecimal byte sequences are interpreted by the operating system in the background. Professional security tools in the industry work similarly with a "black box" logic; they issue an alert when they find a threat but hide the underlying mathematical process and byte parsing steps. Within the scope of this study, we developed a desktop software named "AgKalkan" that parses .pcap files byte by byte directly using the standard file reading capabilities of C++, without requiring external dependencies like libpcap. Our application does not only extract IP and port numbers; it inspects unencrypted packets to capture clear-text credentials and reads TLS SNI extensions in encrypted HTTPS traffic to identify targeted web domains. Thanks to its behavioral rule engine, it reports attacks such as vertical port scanning, SYN flood, and ARP spoofing with a dynamic trust score. Summarizing the results completely through local files, this study is an educational decision support prototype that transparently displays the foundations of complex systems.

Keywords— network security, packet analysis, PCAP parsing, object-oriented design, reverse engineering.

I. INTRODUCTION

At the beginning of our computer engineering education, we all get acquainted with tools like Wireshark during network courses. This software allows us to record and analyze network traffic offline [2]. However, since these ready-made tools parse the packet structure on our behalf, they make it difficult for us to learn what is happening under the hood—namely, how a file is read at the byte level and separated into L2-L4 layers [3]. On the other hand, Intrusion Detection Systems (IDS) and SIEM devices used in the industry also generate alerts by inspecting packet sequences across the network, but most of their alerts may contain false-positive results and hide how they reached this decision from the analyst [4]. This situation poses a serious problem for an engineering student eager to learn cybersecurity. It is impossible to defend or develop a system whose background remains invisible.

When looking at existing analysis tools in the literature, we see that most of them depend on external libraries installed deep within the operating system, such as WinPcap or Npcap,

in order to operate [13]. This dependency completely eliminates the cross-platform portability of the software. Furthermore, traditional systems still operate on a signature-based and rigid logic; whereas to capture modern attacks, behavioral deviations that create anomalies in the system must be monitored [8].

In order to address these shortcomings, we implemented the "AgKalkan" project, which can read binary streams in .pcap files through custom pointer shifts using entirely native C++ code. As shown in Fig. 1, this project aims to prove how analyzed network packets become much more meaningful when inspected from an object-oriented (OOP) perspective through their interconnected communication maps rather than individually [9].

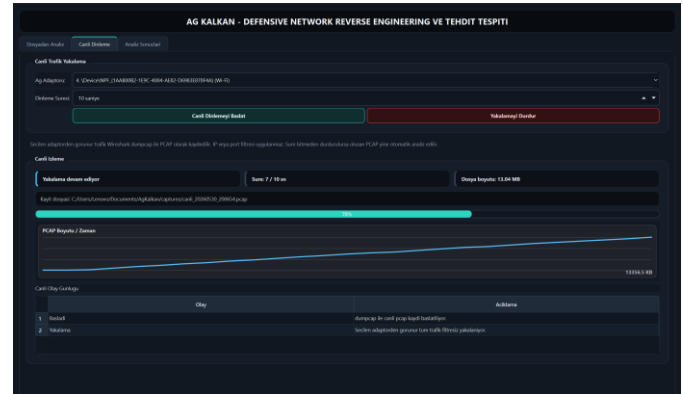


Fig. 1. AgKalkan Desktop UI Layer General View and Live Table Panel.

Instead of sending the captured traffic data to cloud servers, we analyze it completely in an isolated folder (reports) on the user's computer, thereby providing an ethical testing environment in compliance with legal data protection standards (KVKK/GDPR) [14].

II. SYSTEM ARCHITECTURE AND DESIGN

A. Object-Oriented Architecture Selection

While coding our application, we could have taken the easy way out and written all operations inside a single massive file. However, to ensure the sustainability of the code, we chose Object-Oriented Programming (OOP) as our architectural pattern. The project was divided into four main classes. For instance, the PcapOkuyucu class, which is solely responsible for parsing the bytes of the packets, operates

completely isolated from the TehditTespitMotoru class, which extracts meaningful threats from these obtained packets.

B. Network Standards (RFC) and Data Integrity

If you are reading a file in raw binary format, you must strictly comply with international standards (RFC). When performing offset shifts in our code, we based our implementation on the flag structures specified in the RFC 793 standard for TCP connections [15] and the RFC 768 standards for UDP packets [16]. By manually converting the Global Header structure and Little/Big Endian memory alignments inside our C++ code, we prevented the corruption of data integrity [22].

III. THREAT DETECTION METHODS

A. Detected Base Threat Vectors

There are certain terms that frequently appear throughout our paper. For instance, a Man-in-the-Middle (MitM) attack refers to the stealthy interception of communication on a network [5]. Distributed Denial of Service (DDoS) attacks aimed at resource exhaustion represent one of the largest problems straining network architectures [6]. We created specific conditional blocks (if-else states) in our coding to capture such structures.

B. Memory Management and Packet Semicapacity

While inspecting network traffic, thousands of 'packets' and millions of 'bytes' flood the computer's memory within seconds. As frequently discussed in the literature, experiencing RAM leaks during the processing of large-scale network datasets is inevitable [1]. For this reason, we placed a dynamic limit of 5,000 packets on our graphical user interface. Data exceeding the screen capacity was written directly to files on the disk instead of exhausting the memory.

C. Trust Score Calculation Model

We established a mathematical framework for the anomaly level detected in our system. Unlike traditional systems, we designed a flexible trust score formula when determining the risk severity:

$$\text{Trust Score} = 60 + ((\text{Target Port Count} - 20) * \alpha) / \beta \quad (1)$$

Thanks to this equation, movements crossing the threshold are automatically scaled, providing the analyst with a more reliable, dynamic risk percentage between 60 and 95 [4]. How these mathematical correlations reflect onto the interface and the results produced by the rule-based engine are shown in detail in Fig. 2.

Tehdit Tipi/Belirtileri	Risk	Güven	Tahdit Türü	Kaynak IP	Hedef IP	Konut	Açıklama
KRİTİK	95	ARP Spoofing Süphesi	192.168.1.1	-	2 farklı MAC adresi: 08:00:27:00:00:00 ve 08:00:27:00:00:01	Aynı IP adresi farklı MAC adresleriyle görüldü. Bu durum ARP spoofing IP saldırısı veya ağdaki cihaz değişikliği anlamına gelebilir.	
YÜKSEK	65	Port Taraması Süphesi	192.168.1.25	192.168.1.1	25 farklı TCP hedef portu: 21 FTP, 22 Bilgiye, 23 Telnet, 25 SMTP, 53 Bilgiye, 80 Bilgiye, 110 POP3, 135 RPC, 139 NetBIOS, 143 Bilgiye, 389 Bilgiye, 445 SMB, 507 Bilgiye, 593 Bilgiye, 595 Bilgiye, 1433 MySQL, 1521 Oracle DB, 3306 MySQL, 3389 RDP, 5425 Bilgiye, 5900 VNC, 8080 Bilgiye, 8443 Bilgiye, 9000 Bilgiye	Aynı kaynak IP, aynı hedef IP üzerinde çok sayıda farklı TCP porta 5% deneme yapıtı. Bu davranış atak port taraması anlamına gelebilir.	
YÜKSEK	61	SYN Flood Süphesi	192.168.1.40	192.168.1.100	80 SYN paketi, 80 toplam TCP paketi, %100 SYN oranı, hedef port 80	Aynı kaynak hedef port üzerinde yüksek sayıda SYN paketi gönderildi. Tarama amaçlı yüksek sayıda SYN flood veya agresif bağlantı denemesi anlamına gelebilir.	
ORTA	95	Sophisticated Service Portları	192.168.1.25	192.168.1.1	Hafta sonu portları: 21 FTP, 23 Telnet, 25 SMTP, 110 POP3, 139 NetBIOS, 445 SMB, 1433 MySQL, 1521 Oracle DB, 3306 MySQL, 3389 RDP, 5900 VNC	Ekstra servis portları, yoğunlukta aynı hedef olan servis portlarına yönlendirildi. Bu davranış atak amaçlı değişik port taraması ve oturum denemesi ile bağlantı kurulmaya çalışıldığı anlamına gelebilir.	

Fig. 2. Behavioral engine calculated dynamic trust scores and network anomalies screen.

D. Encrypted Traffic (HTTPS) Analysis Constraints

A common major mistake in network analysis is directly attempting to decrypt traffic encrypted with modern protocols like TLS (HTTPS). The mathematics of the network do not allow this. Instead of establishing heavy decryption algorithms [18], we found that extracting the Server Name Indication (SNI) field within the TLS handshake (Client Hello) packet in our C++ code is a much more lightweight and effective way to detect which web domain the data is heading to [17], [20].

IV. APPLICATION LAYER AND FINDINGS

A. Logging and Traceability

In the timestamped TXT summary files generated by the AgKalkan system on the disk, the information of the system user conducting the analysis and the project details are preserved in a fixed log template. This ensures the official validity of the analysis reports within the institution.

B. Application Layer Reverse Engineering

The most critical point of our reverse engineering approach is inspecting the payload of the packet rather than settling for just the packet headers [19]. The C++ state machine we developed scans the word patterns in the payload of an HTTP or FTP packet to capture authentication headers like 'Authorization: Basic' [11]. As shown in Fig. 3, this captured data is sent to our custom Base64 decoder within the system to expose the password as clear-text [7].

HTTP Metot	URL/Yol	Kimlik Bilgisi	Kullanıcı	Parola/Deger	Servis Yorumu
GET	http://test.example.com/login?username=cazla&password=1234567890123	HTTP URL/Form içinde açık metin kimlik bilgi	cazla	123456	Açık HTTP isteği
POST	http://akademik-test.local/panel/giris	HTTP URL/Form içinde açık metin kimlik bilgi	admin	admin123	Açık HTTP isteği
GET	http://basic-auth-test/basic	HTTP Basic Authentication açık metin içeriğidir	demo	demoPass	Açık HTTP isteği
		FTP USER komutu açık metin görüldü	ftpuser		FTP açık metin kontrol trafiği
		FTP PASS komutu açık metin görüldü	ftpuser	ftppass123	FTP açık metin kontrol trafiği
		POP3 USER komutu açık metin görüldü	mailuser		POP3 açık metin posta oturumu
		POP3 PASS komutu açık metin görüldü	mailuser	mailpass456	POP3 açık metin posta oturumu
		IMAP LOGIN komutu açık metin görüldü	imapuser	imappass789	IMAP açık metin posta oturumu
		SMTP AUTH PLAIN vcrisi oluşturuldu	smtpuser	smtppass	SMTP açık metin kimlik doğrulama

Fig. 3. Application layer analysis and clear-text credential discovery extracted from unencrypted HTTP/FTP traffic.

C. Analysis Outputs and Reporting

One of the greatest advantages of the developed project is rescuing all obtained findings from complicated terminal screens and organizing them into a structured layout. The classification logic of our system regarding threats is categorized in TABLE I.

Attack Type	Detected Parameters	Risk Level
Vertical Port Scan	Consecutive 20+ Ports	HIGH
SYN Flood Suspicion	%80+ SYN Rate	HIGH
ARP Spoofing	Multiple Unique MACs	CRITICAL

TABLE I. Classification of Detected Network Threats

The RaporOlusturucu class running in the background generates Excel (xls) and CSV files that can be easily read by machine learning algorithms [12]. The final state of these generated outputs can be observed in the structured Excel report shown in Fig. 4.

Trafik Özeti				
Kategori	Değer	Adet	Bayt	Açıklama
Trafik Yorumu	HTTPS Web Trafiki	16509	16775272	TCP 443 trafiki yoğun. Bu normal web siteleri, uygulama servisi veya bulut bağlantılarından kaynaklanabilir.
Trafik Yorumu	Yerel Ağ ARP Hareketi	36	0	ARP paketleri yerel ağdaki IP-MAC eşleşmelerini gösterir. Tek başına saldırdığıdır; yerel ağ hareketliliğini açıklar.
Trafik Yorumu	Yerel İnternet Yavaş Kullanımı	5198	293384	Yerel ağdaki cihaz yüksek sayıda paket üretimi. Akad internet kullanımı, video izleme veya indirme gibi durumlarda normaldir.
Trafik Yorumu	Çoklu Hedef İnternisi	21	0	Kısa analiz süresinde çok sayıda farklı hedef IP görüldü. Web sayfaları, CDN ve uygulama servisi bu davranışı oluşturabilir.
Protokol Dağılımı	HTTPS	16509	16775272	Paketlerin çoğunluğunun protokol/servis türüne göre dağılımı
Protokol Dağılımı	ARP	36	2124	Paketlerin çoğunluğunun protokol/servis türüne göre dağılımı
Protokol Dağılımı	TCP	14	960	Paketlerin çoğunluğunun protokol/servis türüne göre dağılımı
Protokol Dağılımı	ICMPv6	7	538	Paketlerin çoğunluğunun protokol/servis türüne göre dağılımı
Protokol Dağılımı	DNS	6	614	Paketlerin çoğunluğunun protokol/servis türüne göre dağılımı
Protokol Dağılımı	SSDP	4	848	Paketlerin çoğunluğunun protokol/servis türüne göre dağılımı
Protokol Dağılımı	QUIC	3	209	Paketlerin çoğunluğunun protokol/servis türüne göre dağılımı
Risk Dağılımı	BİGİ	1	0	Tespit motorunun ürettiği uyarı sayısı
En Çok Paket Üreten Kaynak IP	3.169.85.104	11332	16475128	Kaynak IP bazında paket hacmi
En Çok Paket Üreten Kaynak IP	10.37.3.240	5160	290490	Kaynak IP bazında paket hacmi
En Çok Paket Üreten Kaynak IP	10.37.2.1	19	1140	Kaynak IP bazında paket hacmi
En Çok Paket Üreten Kaynak IP	23.109.47.65	16	6225	Kaynak IP bazında paket hacmi
En Çok Paket Üreten Kaynak IP	72.145.35.109	9	3569	Kaynak IP bazında paket hacmi
En Çok Paket Üreten Kaynak IP	10.37.2.215	8	480	Kaynak IP bazında paket hacmi
En Çok Paket Üreten Kaynak IP	91.191.168.225	5	870	Kaynak IP bazında paket hacmi

Fig. 4. Structured Excel summary report output automatically generated by the RaporOlusturucu class.

V. CONCLUSION AND FUTURE WORK

Presented from the perspective of a computer engineering student, this study demonstrates how a clean, object-oriented, and stable network analysis tool can be built using a language with low-level power like C++ without library dependencies. In our synthetic test environments, our program successfully captured vertical port scans seeking vulnerabilities across more than 20 distinct ports [10], [21], SYN Flood attacks consisting of a SYN packet volume greater than 80% [6], and ARP spoofing by detecting different MAC addresses [5] with zero false alarms, exporting them cleanly into Excel tables.

In the future, I plan to elevate this project to the level of a graduation thesis. Integrating a small-scale unsupervised machine learning model trained in Python or a similar environment alongside the existing rule-based engine to automate threat detection capabilities will be the next step of the project.

REFERENCES

- [1] M. H. Erdem and İ. Karayer, "Machine Learning Methods and Network Traffic Analysis," 2022, Journal of Gümüşhane University Institute of Science and Technology. URL: <https://dergipark.org.tr/tr/pub/ngumuh/article/1113956>
- [2] T. Fırlar, "An Examination of Intrusion Detection and Prevention Tools," 2018, Journal of SAU Institute of Science and Technology. URL: <https://dergipark.org.tr/en/download/article-file/667058>
- [3] S. Yılmaz and N. G. Özer, "Network Analysis in the Field of Digital Forensics," 2021, Journal of Süleyman Demirel University Graduate School of Natural and Applied Sciences. URL: <https://dergipark.org.tr/en/download/article-file/747008>
- [4] H. H. İnce and E. A. Sezer, "Comparison of Classification Methods in Intrusion Detection Systems," 2020, Journal of Computer Sciences. URL: <https://dergipark.org.tr/tr/pub/bbd/article/791939>

- [5] İ. Karayer and M. H. Erdem, "Man-In-The-Middle Attack (MITM)," 2022, DergiPark. URL: <https://dergipark.org.tr/en/download/article-file/2703932>
- [6] Y. E. Şahin, "DoS and DDoS Attacks on Internet of Things and Their Detection," 2024, European Journal of Science and Technology. URL: <https://dergipark.org.tr/tr/download/article-file/3666489>
- [7] F. C. Çelik and A. G. Gökçe, "A Study on nDPI Deep Packet Inspection Tool," 2023, International Journal of Information Security Engineering. URL: <https://dergipark.org.tr/tr/pub/tbbmd/article/1253700>
- [8] İ. F. Kılınçer et al., "Intrusion Detection via Anomaly Detection Approach in Computer Networks," 2022, DergiPark. URL: <https://dergipark.org.tr/en/download/article-file/2420612>
- [9] H. İ. Kalkan and S. S. İltir, "Discrete Event System Based Modeling of Cyber Attacks for the Security of Network Systems," 2023, Journal of Information Technologies. URL: <https://dergipark.org.tr/tr/download/article-file/3022952>
- [10] E. A. Sezer, "Attack Types in Network Environment: Attack Scenario Examples," 2023, DergiPark. URL: <https://dergipark.org.tr/en/download/article-file/3081974>
- [11] M. Abbasi et al., "A Novel Proposed Pattern Matching Algorithm for Deep Packet Inspection," 2023, Düzce University Journal of Science and Technology. URL: <https://dergipark.org.tr/tr/pub/dubited/article/1131544>
- [12] Ç. Kaya and O. Yıldız, "Intrusion Detection via Two-Level Hybrid Machine Learning Method," 2019, Marmara Journal of Pure and Applied Sciences. URL: <https://dergipark.org.tr/en/download/article-file/909193>
- [13] T.R. Presidency Digital Transformation Office, "Artificial Intelligence Supported IoT Security Monitoring System," 2024, DergiPark. URL: <https://dergipark.org.tr/en/download/article-file/4950097>
- [14] A. Şahin, "Protection of Personal Data in Online Behavioral Marketing Applications," 2024, DergiPark. URL: <https://dergipark.org.tr/en/download/article-file/4412396>
- [15] IETF, "Transmission Control Protocol," 1981, RFC 793. URL: <https://datatracker.ietf.org/doc/html/rfc793>
- [16] IETF, "User Datagram Protocol," 1980, RFC 768. URL: <https://datatracker.ietf.org/doc/html/rfc768>
- [17] IETF, "The Transport Layer Security (TLS) Protocol Version 1.3," 2018, RFC 8446. URL: <https://datatracker.ietf.org/doc/html/rfc8446>
- [18] C. Xu et al., "Network Intrusion Detection in Encrypted Traffic," 2022, IEEE. URL: <https://ieeexplore.ieee.org/document/9888942>
- [19] G. Engelen et al., "Enhancing Intrusion Detection System (IDS) Through Deep Packet Inspection (DPI)," 2024, IEEE Xplore. URL: <https://ieeexplore.ieee.org/document/10533473>
- [20] Cloudflare, "What Is SNI? How TLS Server Name Indication Works," 2024, Cloudflare Learning Center. URL: <https://www.cloudflare.com/learning/ssl/what-is-sni/>
- [21] G. F. Lyon, "Port Scanning Techniques and Behavioral Analysis," Nmap Network Scanning Reference, 2023. URL: <https://nmap.org/book/man-port-scanning-techniques.html>
- [22] Wireshark Foundation, "PCAP Binary Structure Guide," 2024, Wireshark Developer Docs. URL: https://www.wireshark.org/docs/wsdg_html_chunked/